

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	29 September 2026
Team ID	—
Project Name	AI WeatherWise
Maximum Marks	4 Marks

Technical Architecture:

AI WeatherWise is a weather information and prediction application. Users enter or select a location, the system obtains current and forecast weather data through a weather API, processes the data using backend logic and an AI/ML model, stores relevant information, and presents weather insights through a web interface.

Example: AI-powered weather monitoring and prediction for a selected location

USER		WEATHER API		AI / ML MODEL
Web / Mobile UI	→	Current & forecast data	→	Prediction / analysis
	↓		↓	
FRONTEND HTML • CSS • JavaScript	→	BACKEND Python / Flask	→	DATABASE SQLite / MySQL

Guidelines:

- Include application processes and technology blocks.
- Show the flow from user interface to backend, weather API, AI/ML processing and database.
- Indicate external interfaces such as weather APIs.
- Indicate data storage components.
- Indicate the interface to the machine-learning model.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How the user interacts with the AI WeatherWise application through web/mobile screens.	HTML, CSS, JavaScript
2.	Application Logic-1	Receives location input and manages weather-data requests.	Python / Flask
3.	Application Logic-2	Processes current and forecast weather data and prepares it for display.	Python data processing
4.	Application Logic-3	Generates weather insights and predictions from available weather data.	Python / Machine Learning
5.	Database	Stores user settings, locations and selected weather information.	SQLite / MySQL
6.	Cloud Database	Optional cloud storage for weather records and application data.	Cloud database service
7.	File Storage	Stores configuration files, model files and application assets.	Local Filesystem / Cloud Storage
8.	External API-1	Provides current weather, forecast and location-based weather information.	Weather API
9.	External API-2	Optional supporting API for geocoding/location conversion.	Geocoding API
10.	Machine Learning Model	Analyzes weather features and provides prediction/insight output.	Python ML model
11.	Infrastructure (Server / Cloud)	Runs the frontend, backend, database and ML components.	Local Server / Cloud Deployment

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source frameworks used to build the frontend, backend and ML processing.	Flask, Python ML libraries, JavaScript
2.	Security Implementations	Protect user inputs and application access; validate API requests and avoid exposing secret API keys.	HTTPS, input validation, API-key protection
3.	Scalable Architecture	Separate frontend, backend, API and data/ML layers so components can be scaled independently.	3-Tier Architecture / Cloud Deployment

Table-2: Application Characteristics (continued):

S.No	Characteristics	Description	Technology
4.	Availability	Keep the weather service available by handling API failures gracefully and allowing the backend to retry requests when appropriate.	Cloud hosting / Error handling / Retry logic
5.	Performance	Reduce response time by caching frequently requested weather data and limiting unnecessary API calls.	Caching, optimized API calls, asynchronous requests

References:

1. C4 model – software architecture modelling reference
2. IBM Cloud Architecture – architecture design reference
3. AWS Architecture – cloud architecture reference
4. Weather API documentation – external weather data interface reference